

Project 2: Performance and Cost Case Study

Benchmarking and tuning Spark workloads on NYC TLC data

A measured case study rather than a pipeline build: establishing a baseline, changing one variable at a time, and documenting the performance and cost trade-offs of file layout, cluster configuration and query structure.

This document contains

- Part C: the full Project 2 case study (manual pages 31-39)
- Part A shared set-up and naming standards (manual pages 3-12)
- Parts F and G: finishing, publishing, troubleshooting and the full command reference (manual pages 59-64)

Vivek Pandey — Data Engineer

Azure · Databricks · PySpark · Delta Lake

vp171097.github.io/DevWebsite

PART C

Project 2: Performance and Cost Case Study

Client story: a taxi-analytics job grew from 10 minutes to over an hour and the bill doubled. You'll reproduce it, measure it, fix it one change at a time, prove the answers stay identical, and deliver an audit report. **Duration:** about 10 working days.

C1. Repository layout

```
spark-performance-optimization-case-study/
├── README.md
├── src/perf/
│   ├── __init__.py
│   ├── config.py
│   ├── business_logic.py      # trip rules, fare buckets, airport flag
│   ├── benchmark.py          # timing + metrics harness
│   └── validation.py          # output-equivalence checks
├── notebooks/
│   ├── 00_load_raw_to_bronze
│   ├── 10_v00_baseline
│   ├── 11_v01_native_functions
│   ├── 12_v02_broadcast_join
│   ├── 13_v03_skew_handling
│   ├── 14_v04_projection_and_filters
│   ├── 15_v05_layout_clustering
│   ├── 16_v06_incremental_merge
│   └── 20_results_analysis
├── local/
│   ├── spark_local_config.md
│   ├── skew_experiment.py
│   └── shuffle_partitions_experiment.py
├── scripts/
│   ├── download_tlc.py
│   └── upload_tlc.py
├── tests/  conftest.py  test_business_logic.py  test_validation.py
├── docs/   audit_report.md  audit_report.pdf  results_chart.png  screenshots/
├── data/                                     # git-ignored
└── .env.example  .gitignore  pyproject.toml  requirements-dev.txt  LICENSE
```

► Terminal · repo root

```
cd ~/portfolio
git clone git@github.com:[your-github-username]/spark-performance-optimization-case-study.git
cd spark-performance-optimization-case-study
mkdir -p src/perf notebooks local scripts tests docs/screenshots data/tlc/yellow data/tlc/zones
touch src/perf/{__init__,config,business_logic,benchmark,validation}.py \
      local/{spark_local_config.md,skew_experiment.py,shuffle_partitions_experiment.py} \
      scripts/{download_tlc,upload_tlc}.py \
      tests/{conftest,test_business_logic,test_validation}.py
python3.11 -m venv .venv && source .venv/bin/activate && pip install -r requirements-dev.txt
```

C2. Day 1: data

scripts/download_tlc.py

Purpose	Download monthly yellow-taxi Parquet files and the zone lookup.
Arguments	--year 2023 --months 1-12 --out data/tlc

Logic	1. Expand the months argument (for example 1-12 or 1,2,3) into a list. 2. For each month build the file name yellow_tripdata_[YYYY]-[MM].parquet and the URL https://d37ci6vzurychx.cloudfront.net/trip-data/[file name]. 3. Skip if the local file already exists and is over 1 MB. 4. Download with streaming in 1 MB chunks, a 60-second timeout and 3 retries (2, 4, 8 s); write to a .part file and rename when complete (no half-files). 5. Download the zone lookup from https://d37ci6vzurychx.cloudfront.net/misc/taxi_zone_lookup.csv. 6. For every Parquet file read only the metadata (pyarrow) to get the row count and schema; print a table month / size MB / rows, and warn when a column type differs from January's schema.
Done when	12 files plus the lookup exist; the printed table shows about 3 million rows per month.

► Terminal · repo root

```
python scripts/download_tlc.py --year 2023 --months 1-12 --out data/tlc
# manual equivalent for one month:
curl -L -o data/tlc/yellow/yellow_tripdata_2023-01.parquet \
  https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_2023-01.parquet
curl -L -o data/tlc/zones/taxi_zone_lookup.csv \
  https://d37ci6vzurychx.cloudfront.net/misc/taxi_zone_lookup.csv
```

scripts/upload_tlc.py

Logic	1. Same design as Project 1's uploader: list local files, skip those already present in the Volume, upload with retries, log, summary. Targets: /Volumes/portfolio/perf_raw/tlc/yellow/ and /zones/.
--------------	--

► Terminal · repo root

```
python scripts/upload_tlc.py
# or with the CLI directly:
databricks fs cp -r data/tlc/yellow dbfs:/Volumes/portfolio/perf_raw/tlc/yellow
databricks fs cp data/tlc/zones/taxi_zone_lookup.csv \
  dbfs:/Volumes/portfolio/perf_raw/tlc/zones/taxi_zone_lookup.csv
databricks fs ls dbfs:/Volumes/portfolio/perf_raw/tlc/yellow
```

notebooks/00_load_raw_to_bronze

Purpose	Create consistently typed bronze tables once; every variant reads them.
Logic	1. Define the target schema (put it in config.py): vendor_id int, pickup_at timestamp, dropoff_at timestamp, passenger_count int, trip_distance double, ratecode_id int, store_and_fwd_flag string, pu_location_id int, do_location_id int, payment_type int, fare_amount, extra, mta_tax, tip_amount, tolls_amount, improvement_surcharge, total_amount, congestion_surcharge, airport_fee (all double), source_month string. 2. List the monthly files in the Volume. For each file: read it; lower-case all column names; rename tpep_pickup_datetime → pickup_at, tpep_dropoff_datetime → dropoff_at, VendorID → vendor_id, PULocationID → pu_location_id, DOLocationID → do_location_id, RatecodeID → ratecode_id; add missing columns as nulls; cast to the target types; add source_month from the file name. 3. Union all months by column name and write perf_bronze.trips (overwrite) as plain Delta, with no clustering, partitioning or optimize. 4. Read the zone CSV and write perf_bronze.zones (columns location_id, borough, zone, service_zone). 5. Print row counts per source_month and compare with the download script's table.
Done when	Counts match; DESCRIBE DETAIL shows no clustering columns.

► SQL editor · verify

```
SELECT source_month, COUNT(*) FROM portfolio.perf_bronze.trips GROUP BY 1 ORDER BY 1;
-- numFiles, sizeInBytes, clusteringColumns (should be empty)
DESCRIBE DETAIL portfolio.perf_bronze.trips;
SELECT COUNT(*) FROM portfolio.perf_bronze.zones; -- 265
```

C3. Day 2: business rules, benchmark and validation tools

Output table (per variant)	Grain	Columns
perf_gold.zone_daily_metrics_v0N	pickup_date x pu_zone	pickup_date, pu_zone, pu_borough, trips, revenue, avg_fare, avg_distance, avg_duration_min, tip_rate, low_fare_trips, mid_fare_trips, high_fare_trips, airport_trips
perf_gold.route_top10_v0N	month x route rank 1-10	month, pu_zone, do_zone, trips, avg_fare, route_rank

src/perf/business_logic.py

Contains	• is_valid_trip condition: trip_distance greater than 0 and at most 200; fare_amount 0 or more; dropoff_at after pickup_at; duration under 360 minutes; pickup_at inside the analysed year. • fare_bucket expression: low if fare_amount under 15, mid if 15 to 50 inclusive, high above 50. • is_airport expression: pickup or dropoff zone's service_zone equals "Airports", or the zone name is one of JFK Airport, LaGuardia Airport, Newark Airport. • duration_min expression: seconds between the timestamps divided by 60. • tip_rate definition: sum of tip_amount divided by sum of fare_amount over trips with payment_type = 1 (card), rounded to 4 decimals. • route ranking: within each month, rank routes (pu_zone, do_zone) by trips descending, ties broken by pu_zone then do_zone ascending; keep ranks 1-10. • Output column lists for both tables (validation.py uses them).
Done when	test_business_logic.py covers each boundary: fare 15 is mid; distance 0 is invalid; 200 is valid.

src/perf/benchmark.py

Purpose	Run a variant, time it and store comparable metrics.
Interface	run_benchmark(spark, variant, run_fn, runs=3, notes="") where run_fn executes the full variant and writes its outputs.
Logic	1. Create perf_bench.results if it doesn't exist, with columns: run_id, variant, run_number, started_at, finished_at, duration_sec, rows_in, rows_out_daily, rows_out_routes, output_files, output_bytes, compute_type, spark_version, notes. 2. For run_number 1..runs: drop both output tables of the variant (so nothing is reused). 3. Record started_at (UTC) and a high-resolution start time; call run_fn; record the end time. run_fn must end with writes, so everything has actually executed. 4. Collect rows_in (count of perf_bronze.trips, computed once and cached as a Python value), row counts of the two outputs, and numFiles and sizeInBytes of the daily table (from DESCRIBE DETAIL). 5. compute_type = serverless or local (parameter); spark_version from the session. 6. Append the row to perf_bench.results and print it. 7. After all runs print the median duration.
Done when	Running the baseline produces 3 rows with plausible durations.

src/perf/validation.py

Purpose	Prove each variant gives the same answer as the baseline.
Interface	compare_outputs(spark, baseline_table, candidate_table, keys, measures, tolerance=0.01) returns a result object with passed, details and sample_diffs.

Logic	1. Row counts equal. 2. For each measure column: absolute difference of the sums at most tolerance (integers must be exactly equal). 3. Join on the key columns (full outer): rows only in baseline, rows only in candidate, and rows where any measure differs by more than tolerance. Each count must be 0. 4. Collect up to 20 example mismatching rows into sample_diffs. 5. Return passed only if all checks pass; print a summary line per check.
Done when	test_validation.py shows it detects a changed value, a missing row and an extra row, and passes on identical data.

► Terminal · repo root

```
pytest -q
git add -A && git commit -m "feat: business rules, benchmark and validation tools" && git push
```

C4. Day 3: the baseline (v00)

Write 10_v00_baseline the way a rushed team would. Mark each problem in the notebook with a Markdown cell “ISSUE #n: [name]” so readers can follow the story.

#	Anti-pattern (exact form to write)	Where
1	Python UDFs (row-at-a-time) for fare bucket and the airport flag	Classification
2	Two joins to zones (pickup, dropoff) with automatic broadcast disabled (autoBroadcastJoinThreshold -1, where the setting is allowed)	Enrichment
3	Adaptive execution off (only settable locally or on classic compute; on serverless note that it's managed)	Session
4	Read every column; apply the valid-trip filter only after the joins and UDFs	Start
5	Build the enriched DataFrame and use it for both outputs without persisting it (computed twice)	Outputs
6	Collect the distinct pickup dates to the driver and loop in Python, writing one date at a time in append mode	Daily output
7	Repartition to 2,000 before writing, and no clustering	Write
8	Overwrite all history each run	Write mode

► SQL editor · verify

```
-- after a baseline run
-- note numFiles (expect thousands)
DESCRIBE DETAIL portfolio.perf_gold.zone_daily_metrics_v00;
SELECT variant, run_number, duration_sec FROM portfolio.perf_bench.results ORDER BY started_at;
```

- 1 Run the notebook once end to end to confirm it works, then run the cell that calls run_benchmark for v00 (3 runs).
- 2 Open the **query profile** of the slowest statement (Query history → the query → See query profile), or the Spark UI when running locally. Screenshot: the UDF operator time, the join shuffle size, number of output files.
- 3 Save screenshots as docs/screenshots/v00_[what].png.

C5. Days 4-7: one fix per variant

Notebook	Exact change from the previous variant	Evidence
11_v01_native_functions	Replace both UDFs with the built-in expressions from business_logic.py. Nothing else changes.	UDF operator gone from the plan; time

Notebook	Exact change from the previous variant	Evidence
12_v02_broadcast_join	Wrap the zone lookup in an explicit broadcast hint for both joins; remove the setting that disabled broadcast.	Plan shows broadcast hash join; shuffle bytes
13_v03_skew_handling	Re-enable adaptive execution and skew-join handling (where settable). If the profile still shows a long-tail task on the busiest zones, salt the hot keys: add a salt 0-N to the fact rows for those keys and replicate matching dimension rows N times.	Max vs median task duration
14_v04_projection_and_filters	Select only the 10 needed columns first; apply the valid-trip filter before any join; build the enriched data once into a staging table perf_gold.trips_enriched_v04 and read it for both outputs.	Bytes read; duplicated stages removed
15_v05_layout_clustering	Replace the driver loop with one distributed write; remove the 2,000 repartition; create the output with liquid clustering on (pickup_date, pu_zone); run OPTIMIZE.	numFiles and average file size; a filtered query's time before/after
16_v06_incremental_merge	Process only the newest month: compute its rows, then MERGE into the output on (pickup_date, pu_zone) (update when matched, insert when not). Also recompute route_top10 only for that month.	Incremental run time vs full rebuild

► SQL editor

```
-- used in v05
ALTER TABLE portfolio.perf_gold.zone_daily_metrics_v05 CLUSTER BY (pickup_date, pu_zone);
OPTIMIZE portfolio.perf_gold.zone_daily_metrics_v05;
DESCRIBE DETAIL portfolio.perf_gold.zone_daily_metrics_v05;           -- compare numFiles with v00

-- a "dashboard" query to time before/after layout changes
SELECT pickup_date, SUM(trips), SUM(revenue)
FROM portfolio.perf_gold.zone_daily_metrics_v05
WHERE pu_zone = 'JFK Airport' AND pickup_date BETWEEN '2023-06-01' AND '2023-06-30'
GROUP BY pickup_date ORDER BY pickup_date;

-- history of what each variant wrote
DESCRIBE HISTORY portfolio.perf_gold.zone_daily_metrics_v06;
```

For **every** variant: (1) run run_benchmark 3 times; (2) run compare_outputs against v00 for both output tables; it must pass; (3) screenshot the evidence; (4) commit, for example:

► Terminal · repo root

```
git add -A && git commit -m "perf: v02 broadcast zone lookup (-38% median runtime)" && git push
```

WATCH OUT

Serverless compute manages many Spark settings, so some anti-patterns (AQE off, broadcast threshold) can't be set there. Keep them in the notebook with a note, and show their real effect in the local experiments. State clearly in the README which environment each number comes from.

C6. Day 8: local experiments

► Terminal · repo root

```
cd ~/portfolio/spark-performance-optimization-case-study && source .venv/bin/activate
python local/skew_experiment.py --rows 20000000 --hot-share 0.6
# while it runs, open http://localhost:4040 → Stages → the join stage → Event Timeline
# (screenshot)
python local/shuffle_partitions_experiment.py --rows 10000000
```

local/skew_experiment.py	
Arguments	--rows 20000000 --hot-share 0.6 --keys 1000 --salt 16
Logic	1. Start a local SparkSession: master local[4], driver memory 4g, UI enabled, event log off. Record the settings in spark_local_config.md. 2. Generate the fact table: rows with id, key and value; hot-share of rows get key 1, the rest spread uniformly over keys 2..1000. 3. Generate the dimension: one row per key with a name column. 4. Experiment A: adaptive execution off, broadcast off, shuffle partitions 200 → join, aggregate by name, write to a no-op sink; time it; pause 60 s (input prompt) so you can screenshot the Spark UI. 5. Experiment B: adaptive execution on with skew-join handling on → same; time; pause. 6. Experiment C: adaptive off; salting: add salt = random 0..salt-1 on the fact side for hot keys (0 for others); cross-join the dimension with salts 0..salt-1 for hot keys; join on (key, salt); time; pause. 7. Check that A, B and C produce the same aggregated totals. 8. Print a table: experiment / seconds / speed-up vs A.
Done when	Three Spark UI screenshots show the long-tail task shrinking.

local/shuffle_partitions_experiment.py	
Logic	1. Generate 10 million rows with 100,000 distinct keys. 2. For shuffle partitions 8, 200 and 2,000, with adaptive coalescing off then on: group by key, write Parquet to a temp folder, time it and count the output files. 3. Print the 6-row comparison table.

C7. Days 9-10: results, audit report, README

notebooks/20_results_analysis	
Logic	1. Read perf_bench.results; compute median duration_sec per variant. 2. Compute % change vs v00 and vs the previous variant; add the output_files per variant. 3. Build a bar chart of median runtime per variant, labelled with the % vs baseline; save it as results_chart.png and download it into docs/. 4. Cost estimate: median runtime in hours × an assumed DBU per hour × the list price per DBU from the public pricing page. Put the assumptions in a Markdown cell; label every cost as an estimate. 5. Display the final results table (copy it into the README and the audit report).

Audit report section	Contents
1. Executive summary	3 sentences: problem, what changed, result (runtime and estimated cost)
2. Scope and method	Data size, compute, 3 runs with the median, equivalence validation
3. Findings	Table: issue, evidence (screenshot), impact High/Medium/Low
4. Fixes and results	Results table and chart
5. Recommendations	Monitoring, budgets and alerts, compute policies, predictive optimization, review of other jobs
Appendix	Environment and assumptions

► Terminal · repo root

```
# write docs/audit_report.md, then convert to PDF (pandoc or any Markdown-to-PDF tool)
brew install pandoc # Ubuntu: sudo apt install pandoc
pandoc docs/audit_report.md -o docs/audit_report.pdf
git add -A && git commit -m "docs: results, audit report and README" && git push
```

C8. Azure track: VM right-sizing on Azure Databricks (Virtual Machines)

Free Edition hides the machines. On Azure Databricks, classic clusters run on **Azure Virtual Machines** in your subscription, which is exactly where client money goes. This track adds variant **v07_compute_rightsizing**: the same optimized job on different VM families, spot workers and Photon, with measured runtime and real cost. Prerequisite: A9 done.

Step 1: data in ADLS

► Terminal · repo root

```
azcopy login
azcopy copy "data/tlc/*" "https://$ST.dfs.core.windows.net/landing/tlc/" --recursive
```

► SQL editor · Azure Databricks

```
CREATE EXTERNAL VOLUME IF NOT EXISTS portfolio.perf_raw.tlc
LOCATION 'abfss://landing@stdbxportfolio[SUFFIX].dfs.core.windows.net/tlc';
```

Then run 00_load_raw_to_bronze unchanged (same Volume path).

Step 2: check quota and prices

► Terminal · repo root

```
az vm list-usage -l $LOC -o table | grep -i -E "total regional|DDSv5|EDSv5|LSv3"
```

scripts/vm_prices.py

Purpose	Fetch pay-as-you-go Linux prices per hour for the VM sizes in the test matrix.
Arguments	--region westeurope --sizes Standard_D4ds_v5,Standard_E4ds_v5,Standard_D8ds_v5 --out docs/vm_prices.csv
Logic	1. Call the public Azure Retail Prices API (https://prices.azure.com/api/retail/prices) with a filter on serviceName = Virtual Machines, armRegionName = region, armSkuName = size, priceType = Consumption. 2. Follow the NextPageLink until empty; keep items whose productName doesn't contain Windows and whose skuName doesn't contain Low Priority; separate Spot prices (skuName contains Spot) from regular. 3. Write size, regular_hourly, spot_hourly, currency, effective date to CSV and print a table.
Done when	The CSV has a regular and spot price for each size.

► Terminal · repo root

```
python scripts/vm_prices.py --region westeurope --sizes \
Standard_D4ds_v5,Standard_E4ds_v5,Standard_D8ds_v5
```

Step 3: the test matrix (job clusters)

Config	Driver / workers	Options	Why
c1_general	Standard_D4ds_v5 / 2 × D4ds_v5	on-demand	Typical default
c2_memory	Standard_E4ds_v5 / 2 × E4ds_v5	on-demand	More memory per core for joins
c3_fewer_bigger	Standard_D8ds_v5 / 1 × D8ds_v5	on-demand	Same cores, fewer nodes, less shuffle over network
c4_spot	D4ds_v5 on-demand driver / 2 × D4ds_v5 spot	spot with fallback to on-demand; first on-demand = 1	Cheaper workers

Config	Driver / workers	Options	Why
c5_photon	As c1	Photon runtime enabled	Vectorized engine: higher DBU rate, often shorter runtime
c6_autoscale	D4ds_v5 / autoscale 1-4	auto-termination irrelevant for job clusters	Elasticity vs startup overhead

- 1 Create resources/perf_rightsizing_job.yml in the bundle: one job perf_rightsizing_job with six tasks (one per config), each running notebook 17_v07_compute_rightsizing with parameter config_name, each on its own **job cluster** with the node types and options above, and custom tag project=perf.
- 2 Tasks run one after another (depends-on chain) so they don't compete for quota.
- 3 The notebook runs the v05 logic through run_benchmark with compute_type = config_name and 3 runs.
- 4 Deploy and run with the bundle commands (below).

Step 4: real cost from system tables

► SQL editor · Azure Databricks

```
-- DBUs and list cost per job run (Azure Databricks system tables)
SELECT u.usage_metadata.job_run_id AS run_id, u.sku_name,
       SUM(u.usage_quantity) AS dbus,
       SUM(u.usage_quantity * p.pricing.default) AS dbu_cost_usd
FROM system.billing.usage u
JOIN system.billing.list_prices p
  ON u.sku_name = p.sku_name AND u.usage_start_time >= p.price_start_time
 AND (p.price_end_time IS NULL OR u.usage_start_time < p.price_end_time)
WHERE u.usage_metadata.job_id = '[perf_rightsizing_job id]'
GROUP BY ALL ORDER BY run_id;
```

notebook 20_results_analysis (Azure additions)

Logic	1. Join perf_bench.results (runtime per config) with the DBU cost query above by run id. 2. VM cost per run = runtime hours × (driver price + workers × worker price) from docs/vm_prices.csv (spot price for c4 workers). 3. Total cost = DBU cost + VM cost; add cost per run and cost per month (assuming one run per day). 4. Chart: runtime vs total cost per config (scatter), labelled; the best config is lowest cost within an acceptable runtime. 5. Add a “Compute recommendations” section to the audit report: chosen family, spot policy, Photon yes/no, and a cluster policy that enforces it.
--------------	--

Step 5: enforce it with a cluster policy

Policy setting (perf-cost-guardrails)	Value
Allowed node types	Standard_D4ds_v5, Standard_D8ds_v5, Standard_E4ds_v5 (whatever won the test)
Auto-termination	Fixed at 20 minutes for all-purpose clusters
Max workers	4
Azure availability	Spot with fallback for workers; first on-demand = 1 (the driver)
Custom tags	project and cost-center required (shows up in Azure Cost Management)

► Terminal · repo root

```
databricks bundle deploy -t azure --profile azure
databricks bundle run -t azure perf_rightsizing_job --profile azure
bash policies/create_policy.sh # wraps the readable definition and creates the policy
```

Also open **Azure portal** → **Cost Management** → **Cost analysis**, filter by tag project = perf, group by resource: you'll see the VM, disk and Databricks charges for the managed resource group. Screenshot it for the audit report.

Project 2 acceptance checklist

- ☐ Bronze loaded, unoptimized
- ☐ Rules defined once and tested
- ☐ Benchmark and validation tools tested
- ☐ v00-v06 measured 3× and validated
- ☐ Local experiments with screenshots
- ☐ Results table, chart, cost estimate
- ☐ Audit report PDF
- ☐ README and blog post

PART A

Standards and Shared Set-up

Do this part once. All projects depend on it.

A1. Naming standards

Object	Rule	Examples
GitHub repos	lowercase, hyphen-separated	databricks-ecommerce-lakehouse
Catalog	one for all portfolio work	portfolio
Schemas	[project]_[layer]; prefixes ecom, perf, cdc	ecom_bronze, perf_gold, cdc_silver
Tables	snake_case; gold uses dim_ / fact_ / agg_	orders, dim_customer, agg_daily_sales
Quarantine tables	[table]_quarantine, same schema	ecom_silver.orders_quarantine
Technical columns	leading underscore	_ingested_at, _source_file, _batch_id
Time columns	_at for timestamps, _date for dates	purchased_at, order_date
Pipelines / jobs	[project]_[purpose]_pipeline / _job	ecom_lakehouse_pipeline, ecom_daily_job
Branches	main + feature/[short-name]	feature/silver-layer
Commits	type: summary (feat, fix, docs, test, refactor, chore, perf)	feat: add SCD2 customer dimension

A2. Accounts to create

Account	Used for	Notes
GitHub	Repos, profile, CI/CD	Turn on two-factor authentication
Databricks Free Edition	Pipelines, jobs, dashboards	Sign up with your professional email
Kaggle	Olist dataset (Project 1)	Account → Settings → API → “Create New Token” downloads kaggle.json
Neon (or Supabase)	Hosted PostgreSQL (Project 3)	Alternative: Docker PostgreSQL locally
draw.io / Excalidraw	Diagrams	Browser, no install
Medium / Hashnode / dev.to	Blog posts	Pick one
Microsoft Azure (free account)	Azure tracks and Project 4	Set a budget alert first (A9)

A3. Install local tools

Step 1: Git, Python 3.11 and Java 17

► Terminal · anywhere

```
# macOS (Homebrew)
brew install git python@3.11 openjdk@17
sudo ln -sfn "$(brew --prefix)/opt/openjdk@17/libexec/openjdk.jdk" \
/Library/Java/JavaVirtualMachines/openjdk-17.jdk

# Ubuntu / WSL
sudo apt update && sudo apt install -y git python3.11 python3.11-venv openjdk-17-jdk

# Windows (PowerShell)
winget install Git.Git Python.Python.3.11 EclipseAdoptium.Temurin.17.JDK
```

► Terminal · verify

```
git --version          # expect 2.40+
python3.11 --version   # expect 3.11.x    (Windows: py -3.11 --version)
java -version          # expect 17.x
```

If PySpark later says it cannot find Java, set JAVA_HOME in your shell profile (~/.zshrc or ~/.bashrc):

► Terminal · anywhere

```
# macOS
echo 'export JAVA_HOME=$(/usr/libexec/java_home -v 17)' >> ~/.zshrc && source ~/.zshrc
# Ubuntu / WSL
echo 'export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64' >> ~/.bashrc && source ~/.bashrc
```

Step 2: Git identity and SSH key for GitHub

► Terminal · anywhere

```
git config --global user.name "[Your Name]"
git config --global user.email "[you@yourbrand.com]"
git config --global init.defaultBranch main

ssh-keygen -t ed25519 -C "[you@yourbrand.com]" # press Enter to accept defaults
cat ~/.ssh/id_ed25519.pub                     # copy the output
# Browser: GitHub → Settings → SSH and GPG keys → New SSH key → paste
# expect: "Hi [username]! You've successfully authenticated"
ssh -T git@github.com
```

Step 3: Databricks CLI

► Terminal · anywhere

```
# macOS / Linux / WSL
curl -fsSL https://raw.githubusercontent.com/databricks/setup-cli/main/install.sh | sh
# or macOS: brew tap databricks/tap && brew install databricks
# Windows: winget install Databricks.DatabricksCLI

databricks -v # expect v0.2xx or newer
```

Step 4: VS Code and extensions

- Install VS Code, then the extensions: **Python**, **Databricks**, **YAML**, **Ruff**, **GitHub Actions**, **Rainbow CSV**.
- Open a repo folder with code . from its root.

Step 5: optional tools

► Terminal · anywhere

```
brew install --cask docker # Docker Desktop (Project 3 local Postgres alternative)
# psql client (Ubuntu: sudo apt install postgresql-client)
brew install libpq && brew link --force libpq
brew install gh # GitHub CLI (optional, for secrets and PRs from terminal)
```

Step 6: folder for all projects

► Terminal · anywhere

```
mkdir -p ~/portfolio && cd ~/portfolio
```

A4. Connect the CLI to Databricks

- 1 Sign in to Databricks Free Edition in the browser. Copy the workspace URL from the address bar (everything up to and including .com or .net), for example `https://dbc-1234abcd-5678.cloud.databricks.com`.
- 2 Log in from the terminal with a named profile `portfolio`. A browser window opens; approve it.

► Terminal · anywhere

```
databricks auth login --host https://[your-workspace-url] --profile portfolio
databricks auth profiles # 'portfolio' should show Valid: YES
databricks current-user me --profile portfolio
export DATABRICKS_CONFIG_PROFILE=portfolio # add to ~/.zshrc so every command uses it
```

Fallback if browser login is not available: create a personal access token in **Settings → Developer → Access tokens → Generate new token** (lifetime 90 days) and run:

► Terminal · anywhere

```
databricks configure --token --profile portfolio # paste host, then token
```

A5. Create the catalog, schemas and Volumes

Run either the SQL version (in the Databricks SQL editor) or the CLI version (terminal). Both produce the same result.

► SQL editor

```
CREATE CATALOG IF NOT EXISTS portfolio;

CREATE SCHEMA IF NOT EXISTS portfolio.ecom_landing;
CREATE SCHEMA IF NOT EXISTS portfolio.ecom_bronze;
CREATE SCHEMA IF NOT EXISTS portfolio.ecom_silver;
CREATE SCHEMA IF NOT EXISTS portfolio.ecom_gold;
CREATE SCHEMA IF NOT EXISTS portfolio.ecom_ops;

CREATE SCHEMA IF NOT EXISTS portfolio.perf_raw;
CREATE SCHEMA IF NOT EXISTS portfolio.perf_bronze;
CREATE SCHEMA IF NOT EXISTS portfolio.perf_gold;
CREATE SCHEMA IF NOT EXISTS portfolio.perf_bench;

CREATE SCHEMA IF NOT EXISTS portfolio.cdc_landing;
CREATE SCHEMA IF NOT EXISTS portfolio.cdc_bronze;
CREATE SCHEMA IF NOT EXISTS portfolio.cdc_silver;
CREATE SCHEMA IF NOT EXISTS portfolio.cdc_gold;
CREATE SCHEMA IF NOT EXISTS portfolio.cdc_ops;

CREATE VOLUME IF NOT EXISTS portfolio.ecom_landing.raw;
-- raw files for exploration only
CREATE VOLUME IF NOT EXISTS portfolio.ecom_landing.profiling;
CREATE VOLUME IF NOT EXISTS portfolio.perf_raw.tlc;
CREATE VOLUME IF NOT EXISTS portfolio.cdc_landing.changes;

SHOW SCHEMAS IN portfolio;
SHOW VOLUMES IN portfolio.ecom_landing;
```

► Terminal • alternative to SQL

```
databricks catalogs create portfolio
for s in ecom_landing ecom_bronze ecom_silver ecom_gold ecom_ops \
    perf_raw perf_bronze perf_gold perf_bench \
    cdc_landing cdc_bronze cdc_silver cdc_gold cdc_ops; do
  databricks schemas create "$s" portfolio
done
databricks volumes create portfolio ecom_landing raw MANAGED
databricks volumes create portfolio ecom_landing profiling MANAGED
databricks volumes create portfolio perf_raw tlc MANAGED
databricks volumes create portfolio cdc_landing changes MANAGED
databricks schemas list portfolio
```

WATCH OUT

If Free Edition refuses `CREATE CATALOG`, run `SHOW CATALOGS;`, use the workspace's default catalog instead of `portfolio` everywhere in this manual, and keep all schema names the same. Also note: Free Edition runs on serverless compute with daily quotas and may block outbound internet from notebooks, so every internet-dependent step here runs on your laptop.

A6. Link GitHub to Databricks

- 1 Browser: Databricks → your profile icon → **Settings** → **Linked accounts** → **Git integration** → provider GitHub → link (OAuth app, or a GitHub fine-grained token with repo contents read/write).
- 2 Each time you create a repo: Workspace → your user folder → **Create** → **Git folder** → paste the repo HTTPS URL. Pull changes from the Git folder's branch menu after every local push.

A7. Standard files in every repo

File	Exact contents to put in it
<code>.gitignore</code>	Lines: <code>.env/ __pycache__/* .pyc .env data/ .databricks/* .log .pytest_cache/ .coverage htmlcov/ .DS_Store</code>
<code>.env.example</code>	One setting per line with an empty value: <code>DATABRICKS_CONFIG_PROFILE=portfolio</code> , plus project-specific keys listed in each project
<code>requirements-dev.txt</code>	<code>pyspark==3.5.* · pytest · pytest-cov · chispa · ruff · databricks-sdk · python-dotenv · pandas · pyarrow · requests</code> , plus project extras
<code>pyproject.toml</code>	[project] name and <code>requires-python = ">=3.11"</code> ; [tool.ruff] <code>line-length = 110</code> , <code>target-version py311</code> ; [tool.pytest.ini_options] <code>testpaths = ["tests"]</code> , <code>pythonpath = ["src"]</code>
<code>LICENSE</code>	MIT (choose it when creating the repo on GitHub)
<code>README.md</code>	Structure in Part F

Standard commands you'll use in every repo

► Terminal · repo root

```
# one-time per repo
python3.11 -m venv .venv
source .venv/bin/activate                                # Windows: .venv\Scripts\activate
python -m pip install --upgrade pip
pip install -r requirements-dev.txt                       # then fill in values
cp .env.example .env

# every working session
source .venv/bin/activate
git pull
ruff check . && ruff format --check .                    # lint
pytest -q                                                # tests

# feature workflow
git checkout -b feature/[short-name]
git add -A && git commit -m "feat: [summary]"
git push -u origin feature/[short-name]
# GitHub: open a pull request, merge it, then:
git checkout main && git pull && git branch -d feature/[short-name]
```

A8. The testable-code pattern

All transformation logic lives in plain functions in `src/[package]/transformations/`: each takes a `DataFrame` (plus parameters) and returns a `DataFrame`, with no reading or writing of tables inside. Pipeline files only read, call functions, and declare tables. That's what lets `pytest` run everything locally with a small in-memory `SparkSession`, and it keeps pipeline files short enough for a client to read.

A9. Azure set-up (for the Azure tracks and Project 4)

Every project in this manual has an **Azure track** that swaps free tools for the Azure services clients actually use: **ADLS Gen2**, **Azure SQL Database**, **Cosmos DB**, **Function Apps**, **Virtual Machines**, plus Key Vault. Build each project on Databricks Free Edition first, then do its Azure track in one focused session and delete the resources afterwards. Do this section once, just before your first Azure track.

Azure resource names (used everywhere below)

Resource	Name	Notes
Resource group	rg-dbx-portfolio	Everything lives here; delete it to remove everything
Region	westeurope	Or your nearest region; keep everything in one region
Storage (ADLS Gen2)	stdbxportfolio\$SUFFIX	Containers landing and lake
Access Connector	ac-dbx-portfolio	Managed identity Databricks uses to reach ADLS
Azure Databricks	dbw-portfolio	Trial pricing tier; CLI profile azure
Key Vault	kv-dbx-portfolio-\$SUFFIX	Databricks secret scope kv-portfolio
Azure SQL	sql-dbx-portfolio-\$SUFFIX	Databases shopdb (Project 3) and servingdb (Projects 1 and 4)
Cosmos DB	cosmos-dbx-portfolio-\$SUFFIX	Database appdb; containers customer_profiles, customer_360
Function App	func-dbx-portfolio-\$SUFFIX	Flex Consumption, Python 3.11, system-assigned identity
Virtual Machine	vm-legacy-portfolio	Only in Project 4 (legacy SQL Server + self-hosted runtime)

Resource	Name	Notes
Log Analytics	log-dbx-portfolio	Diagnostic logs (Project 4)

Step 1: variables, login and budget

► Terminal · ~/portfolio/azure.env

```
# put these in ~/portfolio/azure.env and run: source ~/portfolio/azure.env
(SUFFIX = 3-5 random digits)
export SUFFIX=4821
export RG=rg-dbx-portfolio LOC=westeurope
export ST=stdbxportfolio$SUFFIX KV=kv-dbx-portfolio-$SUFFIX
export SQLSRV=sql-dbx-portfolio-$SUFFIX COSMOS=cosmos-dbx-portfolio-$SUFFIX \
  FUNC=func-dbx-portfolio-$SUFFIX
# your Entra object id (after az login)
export ME=$(az ad signed-in-user show --query id -o tsv)
```

► Terminal · repo root

```
az login
az account set --subscription "[subscription name or id]"
source ~/portfolio/azure.env
az group create -n $RG -l $LOC --tags owner=[you] env=lab project=portfolio
az extension add -n databricks
```

Browser: **Cost Management** → **Budgets** → **Add**: scope rg-dbx-portfolio, \$20 per month, alerts at 50/80/100% to your email. Do it before creating anything else.

Step 2: ADLS Gen2, Access Connector and Key Vault

► Terminal · repo root

```
az storage account create -n $ST -g $RG -l $LOC --sku Standard_LRS --kind StorageV2 --hns true \
  --min-tls-version TLS1_2 --allow-blob-public-access false
export ST_ID=$(az storage account show -n $ST -g $RG --query id -o tsv)
az role assignment create --assignee-object-id $ME --assignee-principal-type User \
  --role "Storage Blob Data Contributor" --scope $ST_ID
az storage fs create -n landing --account-name $ST --auth-mode login
az storage fs create -n lake --account-name $ST --auth-mode login

az databricks access-connector create -n ac-dbx-portfolio -g $RG -l $LOC --identity-type \
  SystemAssigned
export AC_ID=$(az databricks access-connector show -n ac-dbx-portfolio -g $RG --query id -o tsv)
export AC_PID=$(az databricks access-connector show -n ac-dbx-portfolio -g $RG --query \
  identity.principalId -o tsv)
az role assignment create --assignee-object-id $AC_PID --assignee-principal-type \
  ServicePrincipal \
  --role "Storage Blob Data Contributor" --scope $ST_ID

az keyvault create -n $KV -g $RG -l $LOC --enable-rbac-authorization true
export KV_ID=$(az keyvault show -n $KV --query id -o tsv)
az role assignment create --assignee-object-id $ME --assignee-principal-type User \
  --role "Key Vault Secrets Officer" --scope $KV_ID
# AzureDatabricks first-party application must read secrets for the Key Vault-backed scope
az role assignment create --assignee 2ff814a6-3304-4ab8-85cb-cd0e6f879c1d \
  --role "Key Vault Secrets User" --scope $KV_ID
# note the DNS name for the scope
az keyvault show -n $KV --query properties.vaultUri -o tsv
```

Step 3: Azure Databricks workspace and Unity Catalog

► Terminal · repo root

```
az databricks workspace create -n dbw-portfolio -g $RG -l $LOC --sku trial
export DBW_URL=https://$(az databricks workspace show -n dbw-portfolio -g $RG --query \
  workspaceUrl -o tsv)
databricks auth login --host $DBW_URL --profile azure
databricks current-user me --profile azure
# vCPU quota for clusters
az vm list-usage -l $LOC -o table | grep -i -E "total regional|DDSV5"
```

- 1 Browser, Azure Databricks: **Catalog → External data → Credentials → Create credential** → Azure Managed Identity → Access Connector ID = the value of \$AC_ID → name cred_adls_portfolio.
- 2 Create the Key Vault-backed secret scope: open [DBW_URL]#secrets/createScope, scope name kv-portfolio, DNS name = vault URI, resource ID = \$KV_ID.
- 3 Compute: create a cluster policy or personal compute with **single node, Standard_D4ds_v5, auto-termination 20 minutes**. Free-trial subscriptions have small vCPU quotas.
- 4 Run the SQL below, then the schema and Volume SQL from A5 (skip its CREATE CATALOG line).

► SQL editor · Azure Databricks

```
CREATE EXTERNAL LOCATION IF NOT EXISTS loc_landing
  URL 'abfss://landing@stdbxportfolio[SUFFIX].dfs.core.windows.net/' WITH (STORAGE CREDENTIAL
    cred_adls_portfolio);
CREATE EXTERNAL LOCATION IF NOT EXISTS loc_lake
  URL 'abfss://lake@stdbxportfolio[SUFFIX].dfs.core.windows.net/' WITH (STORAGE CREDENTIAL
    cred_adls_portfolio);
CREATE CATALOG IF NOT EXISTS portfolio
  MANAGED LOCATION 'abfss://lake@stdbxportfolio[SUFFIX].dfs.core.windows.net/portfolio';
SHOW EXTERNAL LOCATIONS;
```

WATCH OUT

Cost discipline: terminate clusters when you stop, keep auto-termination on, deallocate VMs, and when the Azure track is finished run `az group delete -n $RG --yes --no-wait`. Keep screenshots; they are the portfolio evidence, not the running resources. For the next Azure track, repeat Steps 1–3 (about 20 minutes).

Python packages for Azure tracks (add to requirements-dev.txt when needed)

azure-identity (DefaultAzureCredential uses your az login locally and managed identity in Azure) ·
 azure-storage-file-datalake · azure-keyvault-secrets · azure-cosmos · pyodbc (plus the Microsoft ODBC
 Driver 18 for SQL Server) · azure-functions (inside the Function App project only).

A10. Uploading data into Databricks (Volumes and tables)

In Unity Catalog, **files** live in **Volumes** (governed folders such as /Volumes/portfolio/ecom_landing/raw) and **tables** are Delta tables in a schema. The pipelines in this manual read files from Volumes with Auto Loader, so the main task is getting files into the right Volume folder. Pick the method that fits the job:

#	Method	Best for	Used in
1	Catalog Explorer: Upload to this volume (browser)	A few files, quick exploration	Project 1 profiling files
2	Add data → Create or modify table (browser)	One CSV/JSON/Parquet file straight into a table	Quick sandbox tables
3	Databricks CLI databricks fs cp	Folders and larger files from your laptop	All projects
4	Python SDK scripts (upload_batch.py, upload_tlc.py)	Repeatable, logged uploads	Projects 1 and 2
5	SQL : read_files, COPY INTO	Turning Volume files into tables	Exploration, checks
6	Notebook : %sh curl / dbutils.fs.cp	Downloading from the internet inside Databricks	Project 2 (if internet is allowed)

#	Method	Best for	Used in
7	Azure: azcopy to ADLS Gen2 + external Volume	Client-style landing zones	Azure tracks, Project 4

First: create a sandbox schema for exploration tables

► SQL editor

```
CREATE SCHEMA IF NOT EXISTS portfolio.sandbox COMMENT 'Throw-away tables for exploration';
```

Method 1: upload files to a Volume in the browser

- 1 Left sidebar → **Catalog** → portfolio → ecom_landing → **Volumes** → profiling.
- 2 Optional: **Create directory** (top right) to make a sub-folder, for example olist.
- 3 Click **Upload to this volume**, drag the 9 CSV files from data/raw into the dialog, and click **Upload**.
- 4 Check the files are listed in the Volume page, or run the LIST command below.
- 5 Copy a file's path with the **⋮** → **Copy path** menu when you need it in a notebook.

► SQL editor · verify

```
LIST '/Volumes/portfolio/ecom_landing/profiling/';
```

Method 2: create a table directly from a file (browser)

- 1 Left sidebar → **New (+)** → **Add or upload data** → **Create or modify table**.
- 2 Drop one file (for example olist_orders_dataset.csv).
- 3 Choose catalog portfolio, schema sandbox, table name orders_raw.
- 4 Check the preview: first row as header on; for the reviews file open **Advanced attributes** and enable **multi-line** so comments with line breaks stay in one row.
- 5 Adjust column types if needed, then **Create table**. The result is a managed Delta table you can query at once.

TIP

Use Method 2 only for exploration. Pipelines must read files from Volumes, so production-style flows use Methods 3, 4 or 7.

Method 3: Databricks CLI (from your laptop)

► Terminal · repo root

```
# one file
databricks fs cp data/raw/olist_orders_dataset.csv \
  dbfs:/Volumes/portfolio/ecom_landing/profiling/olist_orders_dataset.csv
# a whole folder (recursive)
databricks fs cp -r data/raw dbfs:/Volumes/portfolio/ecom_landing/profiling/
# one batch into the landing folders the pipeline reads
databricks fs cp -r data/batches/2016_09/orders dbfs:/Volumes/portfolio/ecom_landing/raw/orders/
# replace an existing file (never do this in raw landing folders)
databricks fs cp --overwrite data/raw/olist_orders_dataset.csv \
  dbfs:/Volumes/portfolio/ecom_landing/profiling/olist_orders_dataset.csv
# inspect and clean up
databricks fs mkdir dbfs:/Volumes/portfolio/ecom_landing/profiling/olist
databricks fs ls dbfs:/Volumes/portfolio/ecom_landing/profiling/
databricks fs cat \
  dbfs:/Volumes/portfolio/ecom_landing/profiling/product_category_name_translation.csv
databricks fs rm -r dbfs:/Volumes/portfolio/ecom_landing/profiling/olist
```

Method 4: the upload scripts (repeatable and logged)

► Terminal · repo root

```
python scripts/upload_batch.py --batch 0000_initial      # Project 1: reference data
python scripts/upload_batch.py --next                  # Project 1: next monthly batch
python scripts/upload_batch.py --all --dry-run          # show what would be uploaded
# Project 2: TLC Parquet + zone lookup
python scripts/upload_tlc.py
```

Method 5: SQL on files in Volumes

► SQL editor

```
-- peek at files without creating anything
SELECT * FROM read_files('/Volumes/portfolio/ecom_landing/profiling/olist_orders_dataset.csv',
                        format => 'csv', header => true) LIMIT 10;

-- create a table from a file (types inferred)
CREATE OR REPLACE TABLE portfolio.sandbox.orders_raw AS
SELECT * FROM read_files('/Volumes/portfolio/ecom_landing/profiling/olist_orders_dataset.csv',
                        format => 'csv', header => true, inferSchema => true);

-- multi-line CSV (review comments contain line breaks and quotes)
CREATE OR REPLACE TABLE portfolio.sandbox.reviews_raw AS
SELECT * FROM
  read_files('/Volumes/portfolio/ecom_landing/profiling/olist_order_reviews_dataset.csv',
            format => 'csv', header => true, multiLine => true, escape => '');

-- Parquet folder (Project 2)
SELECT COUNT(*) FROM read_files('/Volumes/portfolio/perf_raw/tlc/yellow/', format => 'parquet');

-- incremental, idempotent load: re-running COPY INTO only loads files it has not loaded before
CREATE TABLE IF NOT EXISTS portfolio.sandbox.orders_copy;
COPY INTO portfolio.sandbox.orders_copy
FROM '/Volumes/portfolio/ecom_landing/raw/orders/'
FILEFORMAT = CSV
FORMAT_OPTIONS ('header' = 'true', 'inferSchema' = 'true')
COPY_OPTIONS ('mergeSchema' = 'true');
SELECT COUNT(*) FROM portfolio.sandbox.orders_copy;
```

Method 6: download inside a notebook (only if outbound internet is allowed)

► Notebook cell (%sh)

```
%sh
mkdir -p /Volumes/portfolio/perf_raw/tlc/zones
curl -L -o /Volumes/portfolio/perf_raw/tlc/zones/taxi_zone_lookup.csv \
  https://d37ci6vzurychx.cloudfront.net/misc/taxi_zone_lookup.csv
ls -lh /Volumes/portfolio/perf_raw/tlc/zones/
```

► Notebook cell (Python)

```
dbutils.fs.ls("/Volumes/portfolio/perf_raw/tlc/zones/")
dbutils.fs.cp("/Volumes/portfolio/ecom_landing/profiling/olist_orders_dataset.csv",
              "/Volumes/portfolio/ecom_landing/profiling/copy/orders.csv") # copy a file
```

Method 7: Azure landing zone (Azure tracks)

Upload with azcopy or the Data Lake SDK to the ADLS container, then expose it through an external Volume (commands in A9, B13 and C8). Databricks then reads it exactly like any other Volume path.

WATCH OUT

Files in the **raw landing** Volumes are immutable: Auto Loader tracks what it has processed and ignores a re-uploaded file with the same name. Upload new data as new files, and keep exploration files in the **profiling** Volume or the sandbox schema. UI uploads are meant for small files; use the CLI or scripts for large ones, and check the current upload limits in the Databricks documentation.

Part A checklist

- ☐ Can upload files to a Volume (A10) and create a table from them
- ☐ Tools installed and verified
- ☐ Git identity and SSH key working
- ☐ CLI profile 'portfolio' valid
- ☐ Catalog, 14 schemas and 3 Volumes exist
- ☐ GitHub linked to Databricks

PART F

Finishing, Publishing and Troubleshooting

F1. README structure

Section	Content
Title + badges	Name; CI badge; tech badges
Business problem	Client story, 2-3 sentences
Results	3-5 bullets with measured numbers
Architecture	Diagram + walkthrough
Data model	Table catalogue and diagram
Design decisions	Choices and alternatives
Screenshots	Graph, lineage, dashboard, CI, alerts
How to run	Every command from this manual, in order
Testing	How to run tests; coverage
Limitations and next steps	Honest list

F2. Publish each project

► Terminal · repo root

```
gh repo edit --description "Databricks lakehouse: Auto Loader, Lakeflow pipelines, SCD2, \
  Unity Catalog, AI/BI" \
  --add-topic databricks --add-topic pyspark --add-topic delta-lake --add-topic \
  data-engineering
git tag -a v1.0 -m "Portfolio release" && git push origin v1.0
gh release create v1.0 --title "v1.0" --notes "First complete version"
```

- 1 GitHub profile → Customize your pins → select the repo.
- 2 Publish the blog post; post on LinkedIn (hook with the result, 4-5 points, diagram; repo link in first comment).
- 3 Add to the Upwork portfolio and the LinkedIn Featured section.

F3. Troubleshooting

Problem	Cause	Fix / command
CLI: "cannot configure default credentials"	No profile selected	export DATABRICKS_CONFIG_PROFILE=portfolio, or add --profile portfolio
Pipeline: ModuleNotFoundError ecom	src not on the path	Set the pipeline root folder to the repo's src, or add the path in the pipeline source settings
Review rows broken across lines	Multi-line text	multiLine plus quote and escape options for order_reviews
Silver drops nearly everything	Rule checked before cleaning, or case mismatch	Apply rules after type_ functions; unit-test the rules
Re-uploaded file ignored	Auto Loader skips processed files	New files need new names; landing is immutable
Need to rebuild everything	Logic changed for existing data	databricks bundle run -t dev [pipeline] --full-refresh-all

Problem	Cause	Fix / command
SCD2 versions out of order	Wrong or non-unique sequence	Sequence by change_ts / seq; break ties with the key
pytest: Java gateway exited	Java missing or JAVA_HOME wrong	java -version; set JAVA_HOME (A3)
CI: bundle validate auth error	Secret names differ	gh secret list; names must match the workflow exactly
psql: SSL required	Neon needs SSL	Keep sslmode=require in PG_CONNECTION
Free Edition quota reached	Heavy compute in one day	Fewer TLC months; run experiments locally; continue next day
Notebook can't reach internet/Postgres	Outbound access restricted	Run on the laptop and upload to a Volume
Azure: 403 AuthorizationPermissionMismatch on ADLS	Control-plane role instead of data role, or role not propagated yet	Assign Storage Blob Data Contributor; wait up to 10 minutes; check the external location's credential
Azure Databricks: cluster fails with QuotaExceeded	vCPU quota for the VM family is too low	az vm list-usage -l \$LOC -o table; use a smaller single-node size or request more quota in the portal (Quotas)
Azure SQL: login failed / client IP not allowed	Firewall rule missing	az sql server firewall-rule create ...; for serverless Databricks compute configure network access
Azure SQL: CREATE USER FROM EXTERNAL PROVIDER fails	Not connected as the Entra admin	sqlcmd -G as the Entra admin set with az sql server ad-admin create
Cosmos DB: 429 Too Many Requests	Writes exceed provisioned RU/s	Enable bulk and throughput control; raise RU/s temporarily
Function App: runs but writes nothing	Managed identity lacks the data role, or wrong app settings	az functionapp identity show; check role assignments and az functionapp config appsettings list
Secret scope: cannot read Key Vault	AzureDatabricks app lacks Key Vault Secrets User	Assign the role to app id 2ff814a6-3304-4ab8-85cb-cd0e6f879c1d
Unexpected Azure bill	Clusters or VM left running	Auto-termination, az vm deallocate, delete the resource group after each track
Benchmark times vary widely	Warm-up and shared resources	3 runs, median, drop outputs before each run

PART G

Command Reference

Every command used in this manual, grouped by tool, for quick lookup.

Git

► Terminal

```
git clone git@github.com:[user]/[repo].git
git status
git pull
git checkout -b feature/[name]
git add -A
git commit -m "type: summary"
git push -u origin feature/[name]
git checkout main
git branch -d feature/[name]
git log --oneline --graph -15
git diff
git restore [file]
git tag -a v1.0 -m "Portfolio release"
git push origin v1.0
```

Python environment and quality

► Terminal

```
python3.11 -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install -r requirements-dev.txt
pip freeze > requirements-lock.txt
ruff check .
ruff check . --fix
ruff format .
pytest -q
pytest -q -k [test_name]
pytest -q --cov=src --cov-report=term-missing
```

Databricks CLI: auth, files, pipelines, jobs

► Terminal

```
databricks auth login --host https://[workspace-url] --profile portfolio
databricks auth profiles
databricks current-user me
databricks catalogs list
databricks schemas list portfolio
databricks volumes list portfolio [schema]
databricks fs ls dbfs:/Volumes/portfolio/[schema]/[volume]/
databricks fs cp [local-file] dbfs:/Volumes/portfolio/[schema]/[volume]/[file]
databricks fs cp -r [local-folder] dbfs:/Volumes/portfolio/[schema]/[volume]/[folder]
databricks fs cp --overwrite [local-file] dbfs:/Volumes/.../[file]
databricks fs rm -r dbfs:/Volumes/portfolio/[schema]/[volume]/[folder]
databricks pipelines list-pipelines
databricks pipelines start-update [pipeline_id]
databricks pipelines get [pipeline_id]
databricks pipelines stop [pipeline_id]
databricks jobs list
databricks jobs run-now [job_id]
databricks jobs list-runs --job-id [job_id] --limit 5
```

Databricks Asset Bundles

► Terminal

```
databricks bundle init # start from a template
databricks bundle generate pipeline --existing-pipeline-id [id] --key [name]
databricks bundle generate job --existing-job-id [id] --key [name]
databricks bundle validate -t dev
databricks bundle deploy -t dev
databricks bundle run -t dev [job_or_pipeline_key]
databricks bundle run -t dev [pipeline_key] --full-refresh-all # rebuild all tables
databricks bundle summary -t dev
databricks bundle destroy -t dev # removes deployed resources
```

SQL editor: inspection and maintenance

► SQL editor

```
SHOW CATALOGS;
SHOW SCHEMAS IN portfolio;
SHOW TABLES IN portfolio.[schema];
SHOW VOLUMES IN portfolio.[schema];
DESCRIBE TABLE EXTENDED portfolio.[schema].[table];
DESCRIBE DETAIL portfolio.[schema].[table];
DESCRIBE HISTORY portfolio.[schema].[table];
SELECT * FROM portfolio.[schema].[table] VERSION AS OF [n];
OPTIMIZE portfolio.[schema].[table];
ALTER TABLE portfolio.[schema].[table] CLUSTER BY ([col1], [col2]);
COMMENT ON TABLE portfolio.[schema].[table] IS '[text]';
ALTER TABLE portfolio.[schema].[table] SET TAGS ('[key]' = '[value]');
GRANT SELECT ON SCHEMA portfolio.[schema] TO `[group]`;
SHOW GRANTS ON SCHEMA portfolio.[schema];
SELECT * FROM event_log('[pipeline_id]') ORDER BY timestamp DESC LIMIT 50;
```

Data download and PostgreSQL

► Terminal

```
kaggle datasets download -d olistbr/brazilian-ecommerce -p data/raw --unzip
curl -L -o [file] \
  https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_[YYYY]-[MM].parquet
docker run --name shopdb -e POSTGRES_PASSWORD=postgres -e POSTGRES_DB=shopdb -p 5432:5432 -d \
  postgres:16
docker start shopdb
docker stop shopdb
docker logs shopdb
set -a && source .env && set +a
psql "$PG_CONNECTION" -f source_db/schema.sql
psql "$PG_CONNECTION" -c "\dt"
```

Project scripts

► Terminal

```
# Project 1
python scripts/prepare_batches.py --raw data/raw --out data/batches [--months N] [--force]
python scripts/generate_customer_updates.py --batch YYYY_MM --pct 2 --seed 42
python scripts/make_bad_rows_batch.py --from-batch 2018_08 --out-batch 9999_bad_rows
python scripts/upload_batch.py --batch [id] | --next | --all [--dry-run]

# Project 2
python scripts/download_tlc.py --year 2023 --months 1-12 --out data/tlc
python scripts/upload_tlc.py
python local/skew_experiment.py --rows 20000000 --hot-share 0.6
python local/shuffle_partitions_experiment.py --rows 10000000

# Project 3
python simulator/seed_source.py --customers 5000 --orders 20000 --seed 7 [--reset]
python simulator/generate_changes.py --duration-min 10 --rate 5 --mix 50,40,10
python extractor/extract_changes.py --env dev|prod [--full] [--dry-run]
```

Azure CLI

► Terminal

```
az login
az account set --subscription "[name or id]"
az group create -n $RG -l $LOC
az group delete -n $RG --yes --no-wait
az storage account create -n $ST -g $RG --kind StorageV2 --hns true --sku Standard_LRS
az storage fs create -n [container] --account-name $ST --auth-mode login
az storage fs file list -f [container] --path [folder] --account-name $ST --auth-mode login \
-o table
az role assignment create --assignee-object-id [id] --assignee-principal-type \
[User|ServicePrincipal] --role "[role]" --scope [id]
az role assignment list -g $RG -o table
az keyvault create -n $KV -g $RG --enable-rbac-authorization true
az keyvault secret set --vault-name $KV -n [name] --value [value]
az databricks workspace create -n dbw-portfolio -g $RG --sku trial
az databricks access-connector create -n ac-dbx-portfolio -g $RG --identity-type SystemAssigned
az sql server create -n $SQLSRV -g $RG --admin-user [user] --admin-password [password]
az sql server ad-admin create -g $RG -s $SQLSRV --display-name [name] --object-id $ME
az sql server firewall-rule create -g $RG -s $SQLSRV -n [rule] --start-ip-address [ip] \
--end-ip-address [ip]
az sql db create -g $RG -s $SQLSRV -n [db] --edition GeneralPurpose --compute-model \
Serverless --family Gen5 \
--capacity 1 --use-free-limit --free-limit-exhaustion-behavior AutoPause
az cosmosdb create -n $COSMOS -g $RG --enable-free-tier true
az cosmosdb sql container create -a $COSMOS -g $RG -d [db] -n [container] \
--partition-key-path "/"[key]" --throughput 400
az functionapp create -g $RG -n $FUNC --storage-account $ST --flexconsumption-location $LOC \
--runtime python --runtime-version 3.11
az functionapp config appsettings set -g $RG -n $FUNC --settings KEY=value
az vm create -g $RG -n [vm] --image [image] --size [size] --admin-username [user]
az vm deallocate -g $RG -n [vm]
az vm list-usage -l $LOC -o table
az deployment group what-if -g $RG -f infra/main.bicep -p infra/parameters.dev.json
az deployment group create -g $RG -f infra/main.bicep -p infra/parameters.dev.json
```

Azure data tools

► Terminal

```
azcopy login
azcopy copy "[local folder]/*" "https://$ST.dfs.core.windows.net/[container]/[path]/" \
--recursive
sqlcmd -S $SQLSRV.database.windows.net -d [db] -G -Q "[query]" # Entra auth
sqlcmd -S $SQLSRV.database.windows.net -d [db] -U [user] -P [password] -i [file.sql]
func init [name] --python
func new --name [function] --template "Timer trigger"
func start
func azure functionapp publish $FUNC
```

GitHub CLI

► Terminal

```
gh auth login
gh repo edit --description "[text]" --add-topic [topic]
gh secret set [NAME]
gh secret set [NAME] --env production
gh secret list
gh pr create --fill
gh pr checks --watch
gh pr merge --squash --delete-branch
gh run list --limit 5
gh run view [run_id] --log-failed
gh release create v1.0 --notes "[text]"
```

Remember: finished and documented beats perfect. Publish Project 1 as soon as its checklist is done, start sending proposals, and build Projects 2 and 3 while you look for clients.